FODS ASSIGNMENT 2

Name – Atharva Shivaji Dabkekar	
Roll no. – 2090139	
PRN - 1251090357	
Division – S.Y A	
Batch – 2	
DATE – 21/07/2026	TIME – 8:00 TO 10:00

QUESTION) Practical 2: Array Operations

Implement one-dimensional array operations including traversal, insertion, deletion, searching, updating, and merging. Compare the execution time of different operations.

ANSWER) We use the clock() function from the <time.h> library to record the CPU time before and after an operation. The execution time is calculated using the formula:

$$\text{Execution Time} = \frac{\text{End Time} - \text{Start Time}}{\text{CLOCKS_PER_SEC}}$$

This gives the time taken by the operation in seconds, which is then displayed on the screen.

ALGORITHM:

Step 1: Start.

Step 2: Read the number of elements (n) of the first array.

Step 3: Read the array elements.

Step 4: Traversal

1. Record the start time using clock().
2. Display all the array elements.
3. Record the end time using clock().
4. Calculate the execution time using:

Execution Time = (End Time − Start Time) / CLOCKS_PER_SEC

5. Display the execution time.

Step 5: Insertion

1. Read the insertion position and value.
2. Record the start time.
3. Shift all elements one position to the right.
4. Insert the new element.
5. Increase the array size by 1.
6. Record the end time.
7. Calculate the execution time.
8. Display the updated array and execution time.

Step 6: Deletion

1. Read the deletion position.
2. Record the start time.
3. Shift the remaining elements one position to the left.
4. Decrease the array size by 1.
5. Record the end time.
6. Calculate the execution time.
7. Display the updated array and execution time.

Step 7: Searching

1. Read the element to be searched.
2. Record the start time.
3. Compare the element with each array element one by one.
4. If found, display its position; otherwise display "Element Not Found".
5. Record the end time.
6. Calculate the execution time.
7. Display the searching time.

Step 8: Updating

1. Read the update position and new value.
2. Record the start time.

3. Replace the existing value with the new value.
4. Record the end time.
5. Calculate the execution time.
6. Display the updated array and execution time.

Step 9: Merging

1. Read the size and elements of the second array.
2. Record the start time.
3. Copy all elements of the first array into the merged array.
4. Copy all elements of the second array after the first array.
5. Record the end time.
6. Calculate the execution time.
7. Display the merged array and execution time.

Step 10: Stop.

Pseudo Code:

```
BEGIN
INPUT n
FOR i ← 0 TO n-1
  INPUT A[i]
END FOR
// Traversal
StartTime ← clock()
FOR i ← 0 TO n-1
  PRINT A[i]
END FOR
EndTime ← clock()
ExecutionTime ← (EndTime - StartTime) / CLOCKS_PER_SEC
PRINT ExecutionTime
// Insertion
```

```
INPUT position
INPUT value
StartTime ← clock()
FOR i ← n DOWNTO position+1
  A[i] ← A[i-1]
END FOR
A[position] ← value
n ← n + 1
EndTime ← clock()
ExecutionTime ← (EndTime - StartTime) / CLOCKS_PER_SEC
PRINT A
PRINT ExecutionTime
// Deletion
INPUT position
StartTime ← clock()
FOR i ← position TO n-2
  A[i] ← A[i+1]
END FOR
n ← n - 1
EndTime ← clock()
ExecutionTime ← (EndTime - StartTime) / CLOCKS_PER_SEC
PRINT A
PRINT ExecutionTime
// Searching
INPUT key
StartTime ← clock()
found ← -1
FOR i ← 0 TO n-1
  IF A[i] = key THEN
    found ← i
```

```
      BREAK
    END IF
  END FOR
  EndTime ← clock()
  ExecutionTime ← (EndTime - StartTime) / CLOCKS_PER_SEC
  IF found = -1 THEN
    PRINT "Element Not Found"
  ELSE
    PRINT "Element Found at Position", found
  END IF
  PRINT ExecutionTime
  // Updating
  INPUT position
  INPUT value
  StartTime ← clock()
  A[position] ← value
  EndTime ← clock()
  ExecutionTime ← (EndTime - StartTime) / CLOCKS_PER_SEC
  PRINT A
  PRINT ExecutionTime
  // Merging
  INPUT n2
  FOR i ← 0 TO n2-1
    INPUT B[i]
  END FOR
  StartTime ← clock()
  FOR i ← 0 TO n-1
    C[i] ← A[i]
  END FOR
  FOR i ← 0 TO n2-1
```

```
    C[n+i] ← B[i]
END FOR
EndTime ← clock()
ExecutionTime ← (EndTime - StartTime) / CLOCKS_PER_SEC
PRINT C
PRINT ExecutionTime
END
```

CODE:

```
#include <stdio.h>
#include <time.h>

#define MAX 1000

// Function to display array elements
void traverse(int arr[], int n)
{
   int i;

   printf("\nArray Elements are:\n");

   for(i=0;i<n;i++)
   {
      printf("%d ",arr[i]);
   }

   printf("\n");
}

// Function to insert an element
void insert(int arr[], int *n, int pos, int value)
```

```
{
   int i;

   for(i=*n;i>pos;i--)
   {
      arr[i]=arr[i-1];
   }

   arr[pos]=value;

   (*n)++;
}

// Function to delete an element
void deleteElement(int arr[], int *n, int pos)
{
   int i;

   for(i=pos;i<(*n)-1;i++)
   {
      arr[i]=arr[i+1];
   }

   (*n)--;
}

// Function to search an element
int search(int arr[], int n, int key)
{
   int i;
```

```
    for(i=0;i<n;i++)
    {
        if(arr[i]==key)
            return i;
    }

    return -1;
}

// Function to update an element
void update(int arr[], int pos, int value)
{
    arr[pos]=value;
}

// Function to merge two arrays
void merge(int arr1[], int n1, int arr2[], int n2, int arr3[])
{
    int i;

    for(i=0;i<n1;i++)
        arr3[i]=arr1[i];

    for(i=0;i<n2;i++)
        arr3[n1+i]=arr2[i];
}

int main()
{
```

```
int arr[MAX],arr2[MAX],merged[MAX];
int n,n2;
int i;
int pos,value,key;
clock_t start,end;
double cpu_time;

printf("Enter number of elements in first array: ");
scanf("%d",&n);

printf("Enter elements:\n");

for(i=0;i<n;i++)
   scanf("%d",&arr[i]);

start=clock();
traverse(arr,n);
end=clock();

cpu_time=((double)(end-start))/CLOCKS_PER_SEC;

printf("Traversal Time = %f seconds\n",cpu_time);

printf("\nEnter position for insertion: ");
scanf("%d",&pos);

printf("Enter value: ");
scanf("%d",&value);
start=clock();
insert(arr,&n,pos,value);
```

```
end=clock();
cpu_time=((double)(end-start))/CLOCKS_PER_SEC;
printf("\nAfter Insertion:\n");
traverse(arr,n);
printf("Insertion Time = %f seconds\n",cpu_time);
printf("\nEnter position for deletion: ");
scanf("%d",&pos);
start=clock();
deleteElement(arr,&n,pos);
end=clock();
cpu_time=((double)(end-start))/CLOCKS_PER_SEC;
printf("\nAfter Deletion:\n");
traverse(arr,n);
printf("Deletion Time = %f seconds\n",cpu_time);
printf("\nEnter element to search: ");
scanf("%d",&key);
start=clock();
int index=search(arr,n,key);
end=clock();
cpu_time=((double)(end-start))/CLOCKS_PER_SEC;
if(index==-1)
   printf("Element Not Found\n");
else
   printf("Element Found at Position %d\n",index);
printf("Searching Time = %f seconds\n",cpu_time);
printf("\nEnter position to update: ");
scanf("%d",&pos);
printf("Enter new value: ");
scanf("%d",&value);
start=clock();
```

```
    update(arr,pos,value);
    end=clock();
    cpu_time=((double)(end-start))/CLOCKS_PER_SEC;
    printf("\nAfter Updating:\n");
    traverse(arr,n);
    printf("Updating Time = %f seconds\n",cpu_time);
    printf("\nEnter number of elements in second array: ");
    scanf("%d",&n2);
    printf("Enter second array elements:\n");
    for(i=0;i<n2;i++)
       scanf("%d",&arr2[i]);
    start=clock();
    merge(arr,n,arr2,n2,merged);
    end=clock();
    cpu_time=((double)(end-start))/CLOCKS_PER_SEC;
    printf("\nMerged Array:\n");
    for(i=0;i<n+n2;i++)
       printf("%d ",merged[i]);
printf("\n");
printf("Merging Time = %f seconds\n",cpu_time);
  return 0;
}
```

OUTPUT:

1)INPUT AND TRANSVERSAL)

```
PS D:\SY_LAB\FODS\LAB\CODE> cd "d:\SY_LAB\FODS\LAB\CODE\" ; if ($?) { gcc 21_7.c -o 21_7 } ; if ($?) { .\21_7 }
Enter number of elements in first array: 10
Enter elements:
42
52
89
78
54
48
56
15
28
99

Array Elements are:
42 52 89 78 54 48 56 15 28 99
Traversal Time = 0.001000 seconds
```

2)INSERTION, DELETION AND SEARCHING)

```
After Insertion:

Array Elements are:
42 52 66 89 78 54 48 56 15 28 99
Insertion Time = 0.000000 seconds

Enter position for deletion: 7

After Deletion:

Array Elements are:
42 52 66 89 78 54 48 15 28 99
Deletion Time = 0.000000 seconds

Enter element to search: 66
Element Found at Position 2
Searching Time = 0.000000 seconds
```

3)UPDATE AND MERGING)

```
Enter position to update: 6
Enter new value: 90

After Updating:

Array Elements are:
42 52 66 89 78 54 90 15 28 99
Updating Time = 0.000000 seconds

Enter number of elements in second array: 5
Enter second array elements:
25
89
78
36
15

Merged Array:
42 52 66 89 78 54 90 15 28 99 25 89 78 36 15
Merging Time = 0.000000 seconds
PS D:\SY_LAB\FODS\LAB\CODE>
```